Concurrencia en Java

7

En esta unidad aprenderás a:

1. **Conocer y diferenciar las arquitecturas multitarea y multihilo.**
2. **Conocer las propiedades y métodos de la clase *Thread* de Java.**
3. **Comprender el ciclo de vida de un hilo y los eventos que provocan su cambio de estado.**
4. **Conocer los mecanismos de sincronización entre hilos (regiones críticas, monitores, semáforos).**

7.1 Multitarea y multihilo

En los sistemas operativos modernos podemos ejecutar varios programas simultáneamente en un ordenador con una única unidad central de proceso (UCP). A esta capacidad se la conoce como *multitarea*. Por **multitarea** entendemos la posibilidad de ejecutar varios procesos (para nosotros, varios programas) sin detener la ejecución de ninguno de ellos de forma perceptible para el usuario. Aunque en cada momento, por cuestiones de arquitectura, en un procesador se ejecuta un único programa, el sistema operativo es capaz de alternar la ejecución de varios programas de forma tan rápida que el usuario tiene una sensación de dedicación exclusiva.

Llamaremos **proceso** a un programa que se ejecuta de forma independiente en un espacio propio de memoria. El sistema operativo multitarea es capaz de ejecutar más de un proceso a la vez. Por ejemplo, en nuestro ordenador podemos estar enviando y recibiendo correos mientras escuchamos nuestra música favorita en MP3. Nuestro ordenador está ejecutando dos procesos independientes, y nosotros tenemos la impresión de que se están ejecutando simultáneamente.

Cada uno de los programas de la figura 7.1 ejecuta su tarea sentencia a sentencia utilizando su memoria local para almacenar los datos o variables necesarias. Esta arquitectura de procesos puede ser suficiente para algunas aplicaciones; sin embargo, en la mayoría de los programas que se ejecutan en nuestro ordenador no es suficiente. Sigamos con el ejemplo del correo electrónico. Para leer el correo electrónico utilizamos un programa que nos permite estar recibiendo y enviando correos mientras redactamos un nuevo correo. Es decir, en la misma aplicación podemos realizar varias tareas de forma simultánea o, lo que es lo mismo, hemos dividido el proceso del correo electrónico en varios subprocesos dentro de la misma aplicación. Estos subprocesos se

Dividimos un proceso en subprocesos = hilos

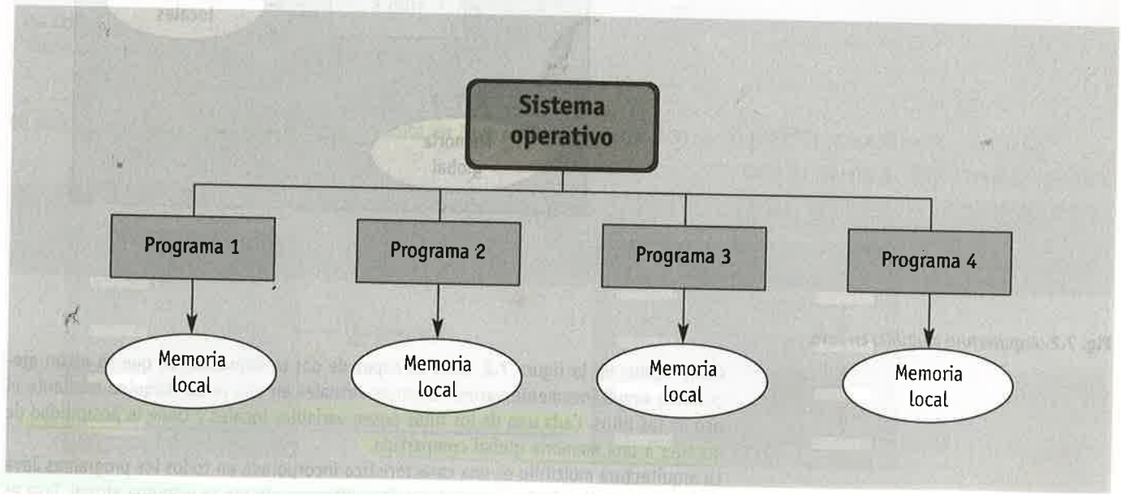


Fig. 7.1. *Arquitectura multitarea.*

conocen con el nombre de **hilos** (del inglés *threads*) y representan un flujo secuencial simple dentro de un proceso.

El concepto de hilo en Java es de vital importancia; por ello, el JDK *(Java Development Kit)* incorpora herramientas y mecanismos para su creación y manipulación. En un entorno multihilo, los hilos aparecen como procesos independientes, con la particularidad de que comparten el mismo espacio de memoria (figura 7.2).

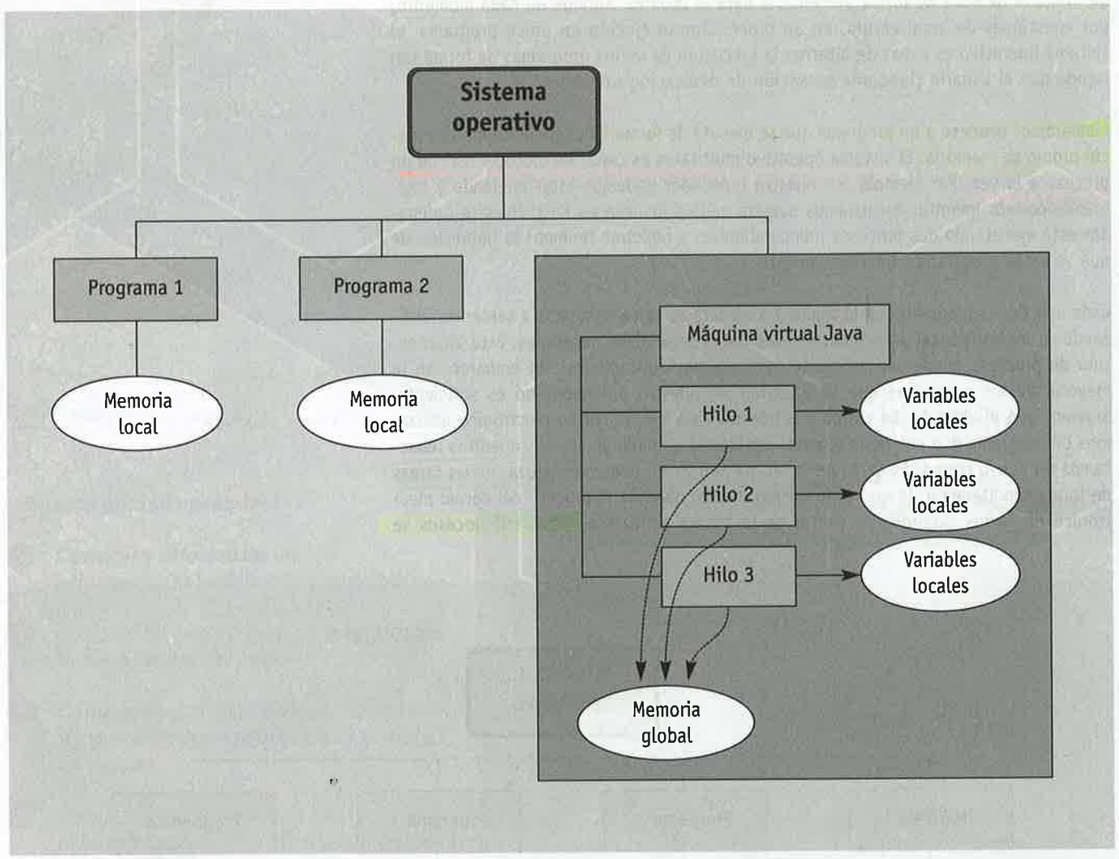


Fig. 7.2. *Arquitectura multihilo en Java.*

Como vemos en la figura 7.2, Java es capaz de dar la impresión de que se están ejecutando simultáneamente varias máquinas virtuales en una única máquina mediante el uso de los hilos. Cada uno de los hilos posee variables locales y tiene la posibilidad de acceder a una memoria global compartida.
La arquitectura multihilo es una característica incorporada en todos los programas Java de forma implícita. Cualquier programa Java interpretado por la máquina virtual Java es un hilo de ejecución. Fíjate en el ejemplo 1.

Ejemplo práctico

1 Programa en Java que muestra cómo cualquier programa en Java se ejecuta mediante un hilo. En el programa se declara un objeto de tipo *Thread* y se recupera la referencia mediante el método *currentThread()*.

```
public class VerificarHiloImplicito
{
public static void main(String args[])
{
   Thread hiloPrincipal = Thread.currentThread();
   System.out.println("-----------------------------------------");
   System.out.println(hiloPrincipal.getName());
   System.out.println(hiloPrincipal.toString());
   System.out.println(hiloPrincipal.activeCount());
   System.out.println("-----------------------------------------");
}
}
```

Java incorpora la clase *Thread* para la creación y manipulación de hilos. Como vemos en el ejemplo 1, estamos declarando un objeto de la clase *Thread* al que llamamos *hiloPrincipal*. En esa misma línea invocamos el método *currentThread*, que devuelve una referencia al hilo que se está ejecutando en ese momento. El programa sigue imprimiendo por la salida estándar la información de este hilo. El resultado de la ejecución de este programa sería:

```
>java VerificarHiloImplicito
-----------------------------------------
main
Thread[main,5,main]
1
-----------------------------------------
```

De manera que cualquier programa que se ejecuta en Java es un hilo (figura 7.3).

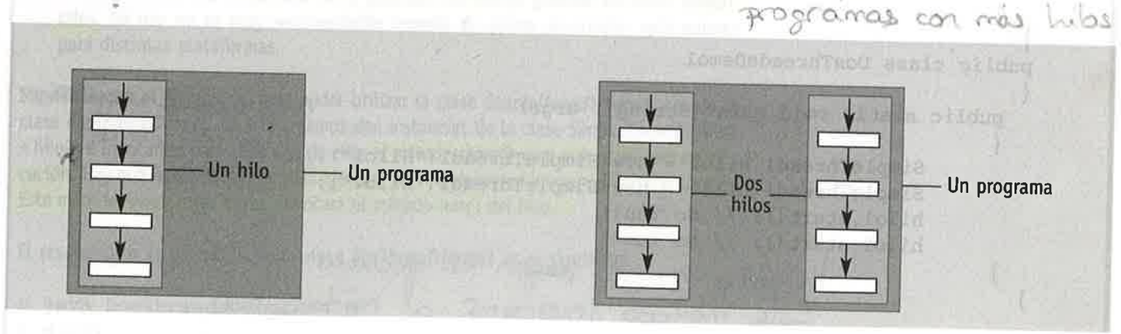


Fig. 7.3. *Cualquier programa en Java tiene al menos un hilo que ejecuta el método* main. *Podemos crear programas con más hilos.*

7.2 Creación de hilos

Podemos crear un hilo de dos formas distintas: mediante la clase *Thread*, o utilizando la interfaz *Runnable*. La interfaz *Runnable* normalmente necesita una instancia de la clase *Thread*, por lo que comenzaremos con esta última clase.

A. Creación de hilos mediante la clase *Thread*

En el ejemplo 2 vamos a crear dos clases: la primera crea una clase *Hilo* a partir de la clase *Thread (SimpleThread1)*, y la segunda es el programa principal *(DosThreadsDemo1)*, que crea dos clases *Hilo*.

Ejemplo práctico

2 Programa en Java que crea dos hilos y los ejecuta. Los hilos escriben diez veces el número de iteración del bucle y su nombre. Al finalizar, los hilos escriben un mensaje de finalización y su nombre.

```
class SimpleThread1 extends Thread
 {
   public SimpleThread1(String nombre)
   {
         super(nombre);
   }
   public void run()
   {
         for (int i = 0; i < 10; i++)
         {
               System.out.println(i + " " + getName());
         }
         System.out.println("Fin! " + getName());
   }
}
public class DosThreadsDemo1
{
   public static void main (String[] args)
   {
         SimpleThread1 hilo1 = new SimpleThread1("Hilo1");
         SimpleThread1 hilo2 = new SimpleThread1("Hilo2");
         hilo1.start(); // No run()
         hilo2.start(); // No run()
   }
}
```


La clase *SimpleThread1* hereda de la clase *Thread*, redefiniendo métodos de ésta. El más importante es el método ***run()***. En este método codificamos la tarea del hilo. En el ejemplo, un hilo es un proceso que imprime diez veces su nombre por la salida estándar. Al heredar de la clase *Thread* tenemos accesibles sus métodos, entre ellos el método *getName()*, que devuelve un *String* con el valor introducido por el constructor de la clase. El constructor de la clase *SimpleThread1* invoca con *super()* al constructor de la clase de la que hereda.

Existen varios métodos constructores para la clase *Thread*:

```
package java.lang;
public class Thread implements Runnable {
    public Thread( );
    public Thread(Runnable target);
    public Thread(ThreadGroup group, Runnable target);
    public Thread(String name);
    public Thread(ThreadGroup group, String name);
    public Thread(Runnable target, String name);
    public Thread(ThreadGroup group, Runnable target, String name);
    public Thread(ThreadGroup group, Runnable target, String name,
                  long stackSize);
    public void start( );
    public void run( );
}
```

Los elementos que queremos destacar de estas definiciones son cuatro:

- ***String name:*** es el nombre del hilo utilizado cuando se imprime información del objeto Hilo. Si no introducimos ningún nombre, por defecto le asignará un valor de "Thread-N", donde *N* es un valor único.
- ***Runnable target:*** de la interfaz *Runnable* nos ocuparemos más adelante. En esta interfaz se puede codificar el conjunto de acciones que queremos que haga un hilo.
- ***Thread group:*** todo hilo pertenece a un grupo. Para el ámbito de las aplicaciones que vamos a tratar aquí, no posee ningún significado. Por defecto, se asigna el grupo del hilo que llama al constructor.
- ***Stack size:*** no es un parámetro que se utilice mucho, pero en algunas ocasiones interesa determinar el tamaño de la pila del hilo donde guardar variables temporales. Su uso no es muy recomendable cuando se quiere desarrollar aplicaciones para distintas plataformas.

Siguiendo con el ejemplo 2, para poder utilizar la clase *SimpleThread1* hemos creado la clase *DosThreadsDemo1*. En ella creamos dos instancias de la clase *SimpleThread1* (hilo1 e hilo2), e invocamos para cada una de ellas el método ***start()*** para que se pongan en ejecución. Hay que hacer notar que para ejecutar un hilo debemos invocar el método *start()*. Este método, entre otras cosas, invocará el método *run()* del hilo.

El resultado de la ejecución de la clase *DosThreadsDemo1* es el siguiente:

```
> java DosThreadsDemo1
0 Hilo1
0 Hilo2
```

```
1 Hilo2
2 Hilo2
3 Hilo2
4 Hilo2
5 Hilo2
6 Hilo2
7 Hilo2
8 Hilo2
1 Hilo1
9 Hilo2
2 Hilo1
Fin! Hilo2
3 Hilo1
4 Hilo1
5 Hilo1
6 Hilo1
7 Hilo1
8 Hilo1
9 Hilo1
Fin! Hilo1
```

Mediante el método *start()* el hilo se registra en un módulo de la máquina virtual de Java llamado **planificador de hilos** *(thread scheduler)*. Éste es el dispositivo que decide cuál es el hilo que se debe ejecutar en cada momento de tiempo. El planificador de hilos va recibiendo peticiones de hilos para poder ejecutar (cuando un hilo se pone en estado *preparado-para-ejecución*). Normalmente, el planificador asigna un tiempo de UCP a cada hilo que lo solicita, utilizando una política de "Primero que entra, primero al que se sirve" (del inglés *First In, First Out: FIFO*). El hecho de invocar el método *start()* no significa que el hilo se ponga en ejecución de forma inmediata; solamente pasa al estado de *preparado-para-ejecución*. Cada hilo compite con los demás para obtener tiempo de UCP para su ejecución. El planificador de hilos asigna los tiempos de ejecución para cada uno de los hilos y, por lo tanto, durante su ciclo de vida pasará tiempo ejecutándose y tiempo en otros estados, como veremos más adelante. En la salida de nuestro ejemplo, el Hilo2 ha podido terminar antes que el Hilo1. Si ejecuta varias veces este programa, es muy probable que en cada caso obtenga distintas salidas. Esto

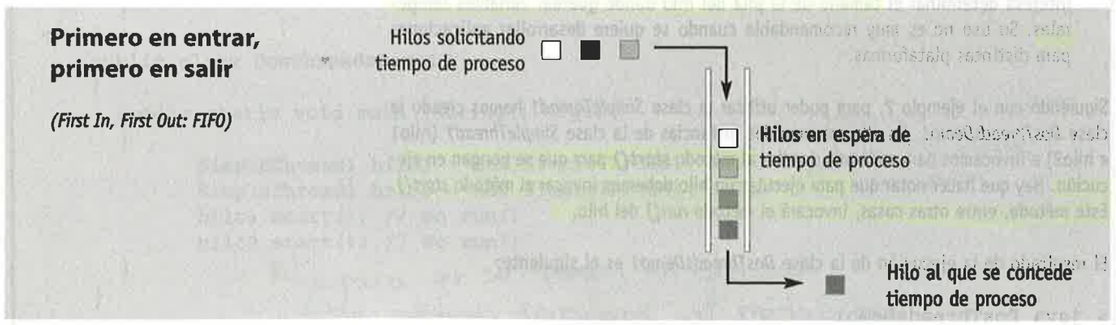


Fig. 7.4. *Política de asignación de tiempo de proceso FIFO* (First In, First Out).

se debe a que el planificador de hilos asigna un determinado tiempo de proceso a cada uno de los hilos. Según el ordenador desde donde se ejecute el programa y la carga que tenga en ese momento, a un hilo le puede dar tiempo de terminar su trabajo (el código del método *run()*) o realizar parte de él antes de consumir el tiempo asignado por el planificador.

B. Creación de hilos mediante la interfaz *Runnable*

Otra forma de crear un hilo es mediante la interfaz *Runnable*. Esta interfaz sólo posee el método *run()*. Aunque a lo largo de esta unidad usaremos la primera forma, en este apartado (ejemplo 3) describimos la codificación de un programa como el del ejemplo 2 utilizando la interfaz *Runnable*.

Ejemplo práctico

3 Programa en Java que hace lo mismo que el del ejemplo 2. En este caso se utiliza la interfaz *Runnable* para la creación de los hilos.

```
class SimpleThreadRunnable implements Runnable
{
    String nombre;
    public SimpleThreadRunnable(String nom)
    {
        this.nombre=nom;
    }
    public void run()
    {
        for (int i = 0; i < 10; i++)
        {
            System.out.println(i + " " + nombre);
        }
        System.out.println("Fin! " + nombre);
    }
}
public class DosThreadsRunnablesDemo
{
    public static void main (String[] args)
    {
        SimpleThreadRunnable hilo1 = new SimpleThreadRunnable("Hilo1");
        SimpleThreadRunnable hilo2 = new SimpleThreadRunnable("Hilo2");
        Thread h1 = new Thread(hilo1);
        Thread h2 = new Thread(hilo2);
        h1.start();
        h2.start();
    }
}
```

En este ejemplo el programa principal crea dos instancias de la clase *SimpleThreadRunnable* que implementa la interfaz *Runnable* (hilo1 e hilo2). También define dos instancias de la clase *Thread* (h1 y h2) y utiliza como parámetros los objetos de la interfaz *Runnable*. El código restante es análogo al del ejemplo 2.

7.3 Terminación de un hilo

Un hilo finaliza su ejecución normalmente cuando termina su método *run()*. Cuando esto ocurre el hilo pasa a un estado especial conocido como *muerto*. Una vez que el hilo llega a este estado, no hay forma de que pueda reiniciarse. La única manera es volver a invocar a su constructor y a su método *start()*.

7.4 Estados de un hilo

Cuando se invoca el método *start()* de un hilo, éste no se ejecuta de forma inmediata, sino que pasa a un estado de *preparado-para-ejecución*. El planificador de hilos se encargará de decidir qué hilo de los que se encuentran en ese estado debe ejecutarse; el hilo seleccionado pasa entonces al estado de *en-ejecución*. En el ciclo de vida de un hilo pueden surgir ciertos eventos que hagan cambiar su estado. Es importante conocer los posibles estados en los que se puede encontrar un hilo:

- **Nuevo**: el hilo ha sido creado, pero todavía no se le ha asignado ninguna tarea, es decir, no se ha invocado el método *start()* del hilo. Una vez activado el hilo, éste pasa al estado de preparado-para-ejecución.
- **Preparado-para-ejecución**: el hilo está a la espera de poder usar la UCP. El planificador de hilos se encargará de pasarlo al estado de en-ejecución.
- **En-ejecución**: es el estado al que aspiran todos los hilos. En este momento el hilo está activo y le ha sido asignado tiempo de UCP. Cuando el planificador de hilos considere que ha consumido su segmento de tiempo, el hilo pasará de nuevo al estado de preparado-para-ejecución y se dará paso a otro hilo preparado-para-ejecución.
- **Bloqueado**: el hilo no puede ejecutarse porque está a la espera de un evento. Una vez que tenga lugar el evento, el hilo pasará al estado de preparado-para-ejecución. Un hilo puede estar bloqueado por las siguientes razones:
  - **Dormido**: el hilo se quedará bloqueado durante un tiempo determinado tras el cual volverá instantáneamente al estado de preparado-para-ejecución.
  - **Esperando**: un hilo se encuentra bloqueado esperando la llegada de un evento. Este evento puede ser la recepción de un mensaje *(notify())*, la finalización de una operación de entrada/salida o el acceso a un método o un objeto sincronizado (como veremos más adelante).
- **Muerto**: un hilo pasa a este estado cuando finaliza su método *run()* o cuando recibe un mensaje *interrupt()*. Un hilo en este estado no puede alcanzar ningún otro estado.

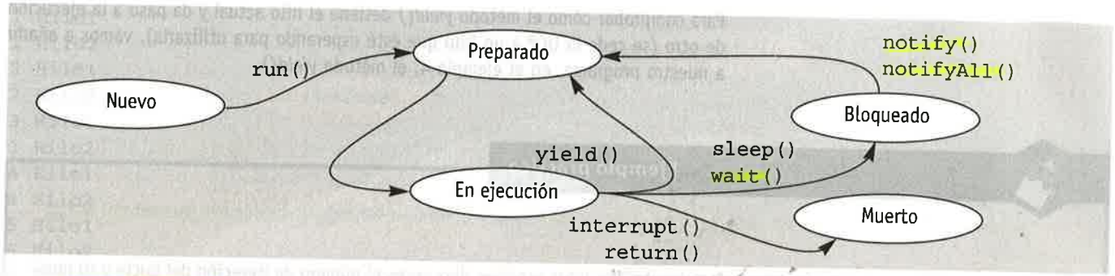


Fig. 7.5. *Estados de un hilo.*

En la figura 7.5 se muestran los estados de un hilo y los eventos que provocan el cambio de estado. Cada una de las formas esféricas de la figura se corresponde con cada uno de los posibles estados en los que puede encontrarse un hilo. En los arcos se etiqueta el método que se debería invocar para provocar la transición de un estado al otro, según el sentido de la flecha. Nótese que el único que no está etiquetado es el arco entre los estados preparado y en-ejecución, ya que es el planificador de hilos el encargado de realizar esta operación. Todos estos métodos se verán en detalle en las secciones siguientes, por lo que se aconseja tener presente este gráfico durante su desarrollo.

7.5 Gestión de los hilos

Con la expresión *gestión de los hilos* nos referimos a las herramientas que incorpora Java para poder cambiar un hilo a sus distintos estados. En este apartado descubriremos los mecanismos para poder pasar del estado de en-ejecución a los demás estados.

A. *Yield*

Un hilo puede ofrecer voluntariamente su tiempo de UCP a otros hilos pasando así al estado de preparado-para-ejecución. Una llamada al método *yield()* provoca que el hilo pase del estado en-ejecución al estado preparado-para-ejecución. Si el planificador de hilos observa que no hay ningún hilo esperando para utilizar la UCP, volverá a cambiar el estado del proceso a en-ejecución, concediéndole tiempo de UCP.

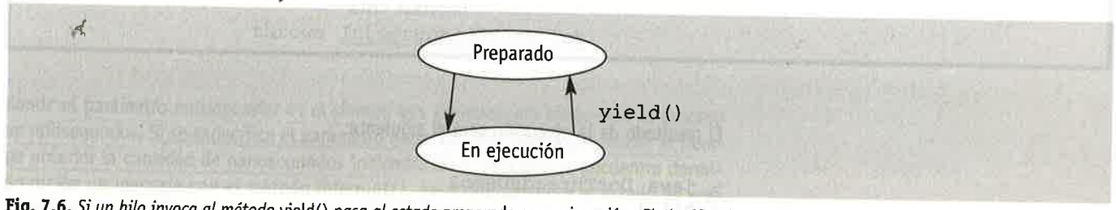


Fig. 7.6. *Si un hilo invoca al método* yield() *pasa al estado* preparado-para-ejecución. *El planificador de hilos se encargará de pasar un hilo que se encuentre en el estado de* preparado-para-ejecución *al estado* en-ejecución.

Para comprobar cómo el método *yield()* detiene el hilo actual y da paso a la ejecución de otro (se cede la UCP a un hilo que esté esperando para utilizarla), vamos a añadir a nuestro programa, en el ejemplo 4, el método *yield()*.

Ejemplo práctico

4 Programa en Java que crea dos hilos y los ejecuta. Los hilos escriben diez veces el número de iteración del bucle y su nombre. En cada iteración, después de escribir su nombre, se bloquean utilizando el método *yield()* y dejan paso al otro hilo.

```
class SimpleThread2 extends Thread
{
  public SimpleThread2(String nombre)
   {
        super(nombre);
   }
   public void run()
   {
        for (int i = 0; i < 10; i++)
        {
              System.out.println(i + " " + getName());
              yield();
        }
        System.out.println("Fin! " + getName());
   }
}

public class DosThreadsDemo2
{
   public static void main (String[] args)
   {
        SimpleThread2 hilo1 = new SimpleThread2("Hilo1");
        SimpleThread2 hilo2 = new SimpleThread2("Hilo2");
        hilo1.start();
        hilo2.start();
   }
}
```

El resultado de la ejecución será el siguiente:

```
> java DosThreadsDemo2
0 Hilo1
0 Hilo2
1 Hilo1
1 Hilo2
2 Hilo1
2 Hilo2
3 Hilo1
3 Hilo2
4 Hilo1
4 Hilo2
5 Hilo1
5 Hilo2
6 Hilo1
6 Hilo2
7 Hilo1
7 Hilo2
8 Hilo1
8 Hilo2
9 Hilo1
9 Hilo2
Fin! Hilo1
Fin! Hilo2
```

Cada vez que un hilo imprime su correspondiente número, su nombre da paso al otro hilo. El planificador de hilos asigna un tiempo de UCP a cada uno de los hilos. En nuestro ejemplo sólo tiene que realizar una operación de escritura en la salida estándar del número de iteración y su nombre. El tiempo concedido por el planificador de hilos a cada hilo es suficiente para poder realizar esta tarea. Por esta razón vemos cómo se van alternando los mensajes de ambos hilos.

B. Dormido *(sleep)*

Cuando un hilo utiliza el método *sleep()* pasa al estado de *bloqueado* durante una cantidad de tiempo definida. El método *sleep()* viene definido por:

```
public static void sleep(long milisegundos)
                  throws InterruptedException

public static void sleep(long millis,
                         int nanos)
                  throws InterruptedException
```

donde el parámetro *milisegundos* es el tiempo que permanecerá bloqueado el proceso en milisegundos. Si se especifica el parámetro *nanosegundos*, habrá que añadir al tiempo anterior la cantidad de nanosegundos indicada. Si un hilo que se encuentra dormido recibe un mensaje con el método *interrupt()*, pasa directamente al estado de preparado-para-ejecución, y cuando pasa al estado en-ejecución, ejecuta el código del manejador de la excepción *InterruptedException*.

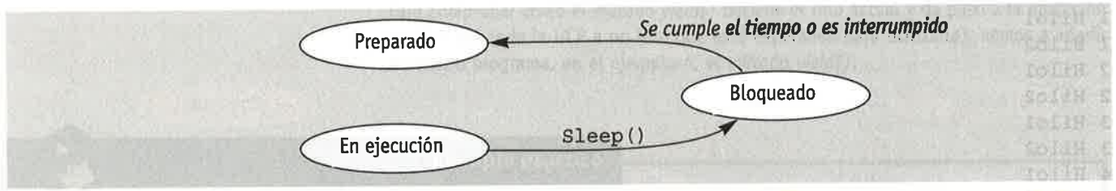


Fig. 7.7. *Cuando un proceso invoca el método* sleep() *se bloquea hasta que transcurre el tiempo* t *indicado en el parámetro del método o le llega una interrupción.*

Ejemplo práctico

5 Programa en Java que crea dos hilos y los ejecuta. Los hilos escriben diez veces el número de iteración del bucle y su nombre. En cada iteración, después de escribir su nombre, se bloquean durante un tiempo aleatorio de segundos y después vuelven a estar disponibles para su ejecución.

```
class SimpleThread3 extends Thread
{
    public SimpleThread3(String nombre)
    {
        super(nombre);
    }
    public void run()
    {
        for (int i = 0; i < 10; i++)
        {
            System.out.println(i + " " + getName());
            try
            {
                sleep((long)(Math.random() * 1000));
            } catch (InterruptedException e)
            {
            }
        }
        System.out.println("Fin! " + getName());
    }
}
public class DosThreadsDemo3
{
    public static void main (String[] args)
    {
        SimpleThread3 hilo1 = new SimpleThread3("Hilo1");
        SimpleThread3 hilo2 = new SimpleThread3("Hilo2");
        hilo1.start();
        hilo2.start();
    }
}
```


El resultado de la ejecución es el siguiente:

```
0 Hilo1
0 Hilo2
1 Hilo2
1 Hilo1
2 Hilo1
2 Hilo2
3 Hilo1
3 Hilo2
4 Hilo2
5 Hilo2
4 Hilo1
5 Hilo1
6 Hilo1
6 Hilo2
7 Hilo2
7 Hilo1
8 Hilo2
8 Hilo1
9 Hilo1
Fin! Hilo1
9 Hilo2
Fin! Hilo2
```

Con el método *Math.random()* obtenemos un valor entre 0,0 y 0,1, que al multiplicarlo por 1.000 produce una espera entre 0 y 1 segundos.

C. *Join*

Un hilo puede invocar el método *join()* de otro hilo con el objeto de quedarse bloqueado a la espera de que termine el segundo hilo. Es una buena forma de crear una

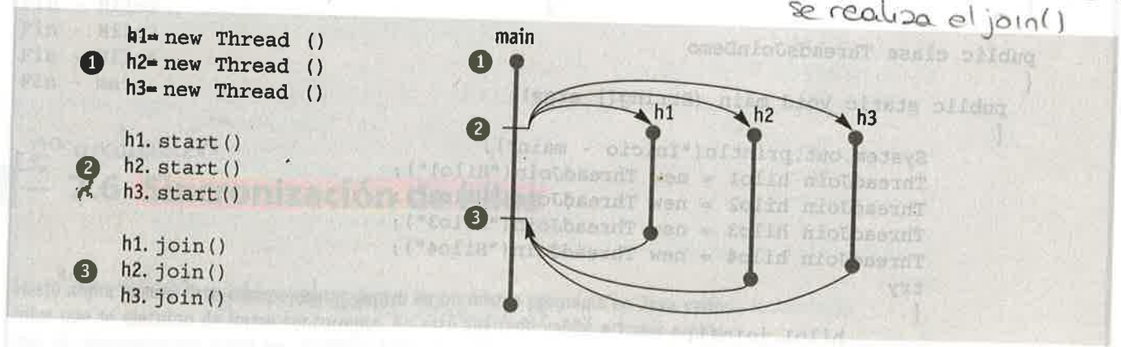


Fig. 7.8. *Un ejemplo en el que el programa principal* main *crea tres hilos (1), invoca el método* start() *de los tres hilos (2) y se queda bloqueado esperando a que terminen los tres hilos usando el método* join() *(3).*

estructura de hilos. Puede existir un hilo padre que cree unos cuantos hilos hijo, los arranque y espere a que éstos terminen su ejecución. Un hilo bloqueado que ha invocado el método *join()* de otro hilo puede recibir un mensaje mediante la llamada *interrupt()*, por lo que es necesario tratar la excepción *InterruptedException*.

Ejemplo práctico

6 Programa en Java que crea cuatro hilos, los ejecuta y espera a que éstos terminen. Los hilos escriben cinco veces el número de iteración del bucle y su nombre. En cada iteración, después de escribir su nombre, se bloquean durante un segundo y vuelven a estar disponibles para su ejecución.

```
class ThreadJoin extends Thread
  {
    public ThreadJoin(String nombre)
    {
          super(nombre);
          start();
    }
    public void run()
    {
          try
          {
                for (int i = 0; i < 4; i++)
                {
                      System.out.println(getName() + " " + i);
                      Thread.sleep(1000);
                }
                System.out.println("Fin - " + getName());
          } catch (InterruptedException e){}
    }
}

public class ThreadsJoinDemo
{
  public static void main (String[] args)
  {
        System.out.println("Inicio - main");
        ThreadJoin hilo1 = new ThreadJoin("Hilo1");
        ThreadJoin hilo2 = new ThreadJoin("Hilo2");
        ThreadJoin hilo3 = new ThreadJoin("Hilo3");
        ThreadJoin hilo4 = new ThreadJoin("Hilo4");
        try
        {
              hilo1.join();
              hilo2.join();
              hilo3.join();
              hilo4.join();
              System.out.println("Fin - main");
        } catch (InterruptedException e)
        {
        }
  }
}
```


El resultado de la ejecución es el siguiente:

```
Inicio - main
Hilo1 0
Hilo2 0
Hilo3 0
Hilo4 0
Hilo1 1
Hilo2 1
Hilo3 1
Hilo4 1
Hilo1 2
Hilo2 2
Hilo3 2
Hilo4 2
Hilo1 3
Hilo2 3
Hilo3 3
Hilo4 3
Fin - Hilo1
Fin - Hilo2
Fin - Hilo3
Fin - Hilo4
Fin - main
```

7.6 Sincronización de hilos

Hasta ahora hemos visto cómo ejecutar dentro de un mismo programa en Java varios hilos que se ejecutan de forma concurrente. En este apartado vamos a tratar problemas de sincronización entre los distintos hilos que se ejecutan concurrentemente. Cuando hablamos de **sincronización** nos referimos al hecho de que dos o más hilos compiten por la obtención de un recurso.

A. Regiones críticas *(synchronized)*

Para comenzar vamos a plantear el problema que tendríamos si varios hilos accedieran de forma concurrente y no sincronizada a una información que pudieran consultar y modificar. El siguiente ejemplo crea cuatro hilos que acceden a un recurso compartido llamado *Contador*.

Ejemplo práctico

7 Programa en Java que crea cuatro hilos que acceden a una variable compartida que es un contador. Cada uno de los hilos accede ~~cuatro~~ veces al contador y aumenta en una unidad su valor. En este programa no usamos ninguna herramienta de sincronización.

```
/**************************************************************
 Clase ContadorTest
 Creamos cuatro hilos que comparten el recurso compartido
 Contador.
 **************************************************************/

public class ContadorTest
{
    public static void main (String[] args)
    {
        Contador cont = new Contador();
        ContadorHilo hilo1 = new ContadorHilo("Hilo1",cont);
        ContadorHilo hilo2 = new ContadorHilo("Hilo2",cont);
        ContadorHilo hilo3 = new ContadorHilo("Hilo3",cont);
        ContadorHilo hilo4 = new ContadorHilo("Hilo4",cont);

        hilo1.start();
        hilo2.start();
        hilo3.start();
        hilo4.start();
        try
        {
            hilo1.join();
            hilo2.join();
            hilo3.join();
            hilo4.join();
        } catch (InterruptedException e)
        {
            System.out.println("Interrupción del hilo...");
        }
```



```
        System.out.println("Terminamos el programa...");
    }
}

/**************************************************************
Clase Contador
Recurso compartido por los procesos. Sección crítica.
**************************************************************/

class Contador
{
    protected int contador=0;

    //Devuelve el valor actual del contador
    public int getContador()
    {
        return this.contador;
    }

    //Establece el valor actual del contador
    public void setContador(int val)
    {
        this.contador=val;
    }
}

/**************************************************************
Clase ContadorHilo
Cada hilo accederá ~~valores~~ veces al recurso compartido Contador
y lo incrementará en 1, durmiendo un tiempo aleatorio en cada
iteración
**************************************************************/

class ContadorHilo extends Thread
{
    Contador cont;
    ContadorHilo(String p_nombre, Contador p_cont)
    {
        super(p_nombre);
        this.cont=p_cont;
    }
    //Método para incrementar el contador
    public void run()
    {
        for (int i=0;i<3;i++)
        {
            try
            {
```

```
                    int a = cont.getContador();
                    sleep((long)(Math.random() * 10));
                    cont.setContador(a+1);
                    System.out.println(getName()
                     +" - contador: "+cont.getContador());

                    sleep((long)(Math.random() * 10));
                } catch (InterruptedException e)
                {
                  System.out.println("Interrupción del hilo...");
                }
        System.out.println("Fin "+getName()+"...");
    }
}
```

La salida del programa sería la siguiente:

```
Hilo2 - contador: 1
Hilo1 - contador: 1
Hilo4 - contador: 1
Hilo3 - contador: 1
Hilo2 - contador: 2
Hilo1 - contador: 3
Hilo2 - contador: 4
Hilo4 - contador: 4
Hilo3 - contador: 4
Hilo1 - contador: 4
Fin Hilo2...
Hilo3 - contador: 5
Fin Hilo3...
Fin Hilo1...
Hilo4 - contador: 5
Fin Hilo4...
Terminamos el programa...
```

El programa no funciona correctamente, ya que los hilos no se sincronizan a la hora de acceder al recurso compartido (Contador). Si los hilos estuvieran sincronizados, el contador debería alcanzar el valor de 12, es decir, el resultado de 4 hilos aumentando 3 veces el contador. Una explicación de lo ocurrido se encuentra en la figura 7.9. En definitiva, debemos realizar las operaciones de lectura y actualización de forma sincronizada.

Introducimos entonces el concepto de **región crítica**. Java incorpora mecanismos para hacer que el acceso a un recurso compartido se haga de forma exclusiva. Es decir, vamos a indicar la parte del código que sólo puede ser ejecutada por un único hilo en un momento dado. Si un hilo intenta acceder a ese código cuando está siendo ejecutado por otro hilo, éste se bloqueará hasta que el segundo termine su ejecución. Obviamente,

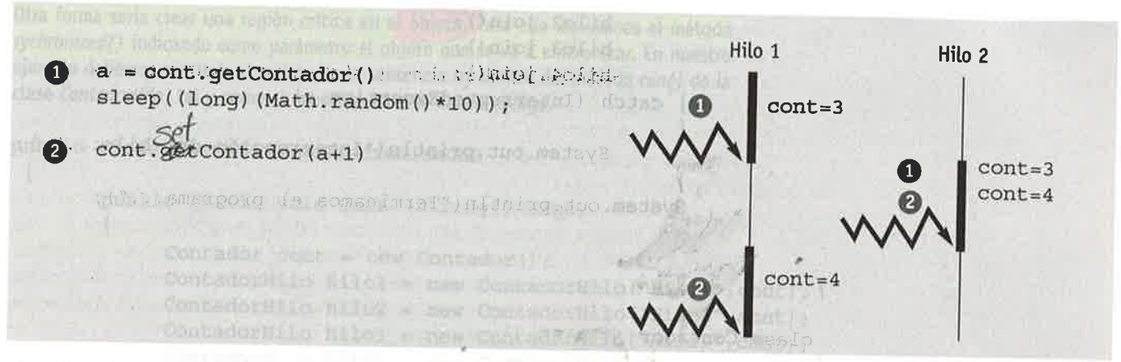


Fig. 7.9. *Los dos hilos ejecutan el mismo código y acceden a un objeto compartido* cont. cont *es un contador que tiene dos métodos:* getContador(), *para obtener el contenido actual del contador, y* setContador(), *para actualizar el contenido del contador. Durante el tiempo de proceso asignado al* Hilo1 *sólo le da tiempo a ejecutar el código indicado por (1) (por ejemplo, un valor de 3) y guarda en su variable local* a *el valor del contador. Sin embargo, al* Hilo2 *le da tiempo de leer el contador (1) y aumentarlo en una unidad (2); es decir, el contador tiene un valor de 4. Cuando vuelve a tener tiempo el* Hilo1, *sigue por donde se había quedado y actualiza el contador con un valor de 4. De esta forma, el contador sólo se ha incrementado en una unidad, cuando tendría que haberlo hecho en dos.*

la parte del código que nos interesa que se ejecute de forma exclusiva será aquella donde se consulta y aumenta el valor del contador.

La región crítica la podemos crear en el método o en el objeto. Para indicar que un método es una región crítica debemos especificarlo en su definición con la palabra reservada *synchronized*.

En el ejemplo 7 podemos sustituir los métodos *getContador()* y *setContador()* por el método *incContador()*, siendo éste un método sincronizado. Además, debemos modificar la clase *ContadorHilo*, que debe invocar este nuevo método. El ejemplo 7 modificado sería el siguiente:

```
public class ContadorTestSincronizado1
 {
       public static void main (String[] args)
       {
            Contador cont = new Contador();
            ContadorHilo hilo1 = new ContadorHilo("Hilo1",cont);
            ContadorHilo hilo2 = new ContadorHilo("Hilo2",cont);
            ContadorHilo hilo3 = new ContadorHilo("Hilo3",cont);
            ContadorHilo hilo4 = new ContadorHilo("Hilo4",cont);

            hilo1.start();
            hilo2.start();
            hilo3.start();
            hilo4.start();
            try
            {
                  hilo1.join();
```

```
                hilo2.join();
                hilo3.join();
                hilo4.join();
            } catch (InterruptedException e)
            {
                System.out.println("Interrupción del hilo..." );
            }
            System.out.println("Terminamos el programa...");

        }
}

 class Contador
{
        protected int contador=0;

        //Incrementar el valor actual del contador
        public synchronized int incContador(int val)
        {
            this.contador+=val;
            try
            {
                Thread.sleep((long)(Math.random() * 100));
            } catch (InterruptedException e)
            {
                System.out.println("Interrupción del hilo..." );
            }
            return this.contador;
        }
}

class ContadorHilo extends Thread
{
        Contador cont;
        ContadorHilo(String p_nombre, Contador p_cont)
        {
            super(p_nombre);
            this.cont=p_cont;
        }
        //Método para incrementar el contador
        public void run()
        {
            for (int i=0;i<3;i++)
            {
                System.out.println(getName()
                    + " - contador: " + cont.incContador(1));

            }
            System.out.println("Fin "+getName()+"...");
        }
}
```

método

método sincronizado

Otra forma sería crear una región crítica en el objeto. Para ello utilizamos el método *sychronized()* indicando como parámetro el objeto que se va a sincronizar. En nuestro ejemplo debemos sustituir el cuerpo de la sentencia *try* dentro del método *run()* de la clase *ContadorHilo*, tal y como se muestra en este nuevo código.

```
public class ContadorTestSincronizado2
 {
        public static void main (String[] args)
        {
            Contador cont = new Contador();
            ContadorHilo hilo1 = new ContadorHilo("Hilo1",cont);
            ContadorHilo hilo2 = new ContadorHilo("Hilo2",cont);
            ContadorHilo hilo3 = new ContadorHilo("Hilo3",cont);
            ContadorHilo hilo4 = new ContadorHilo("Hilo4",cont);

            hilo1.start();
            hilo2.start();
            hilo3.start();
            hilo4.start();
            try
            {
                hilo1.join();
                hilo2.join();
                hilo3.join();
                hilo4.join();
            } catch (InterruptedException e)
            {
                System.out.println("Interrupción del hilo..." );
            }
            System.out.println("Terminamos el programa...");
        }
}
 class Contador
{
        protected int contador=0;

        //Devuelve el valor actual del contador
        public int getContador()
        {
            return this.contador;
        }

        //Establece el valor actual del contador
        public void setContador(int val)
        {
            this.contador=val;
        }
}
class ContadorHilo extends Thread
{
        Contador cont;
```

objeto que vamos a establecer como sincronizado

```
ContadorHilo(String p_nombre, Contador p_cont)
{
     super(p_nombre);
     this.cont=p_cont;
}
//Método para incrementar el contador
public void run()
{
     for (int i=0;i<3;i++)
     {
          try
          {
          synchronized (cont)
          {
               int a = cont.getContador();
               sleep((long)(Math.random() * 10));
               cont.setContador(a+1);
               System.out.println(getName()
              +" - contador: "+cont.getContador());

               sleep((long)(Math.random() * 10));
          }
          } catch (InterruptedException e)
          {
            System.out.println("Interrupción del hilo...");
          }
        System.out.println("Fin "+getName()+"...");
     }
  }
}
```

objeto sincronizado

En ambos casos obtendremos el resultado deseado. Es decir, el contador con un valor de 12.

```
Hilo1 - contador: 1
Hilo2 - contador: 2
Hilo3 - contador: 3
Hilo4 - contador: 4
Hilo1 - contador: 5
Hilo2 - contador: 6
Hilo3 - contador: 7
Hilo4 - contador: 8
Hilo1 - contador: 9
Fin Hilo1...
Hilo2 - contador: 10
Fin Hilo2...
Hilo3 - contador: 11
Fin Hilo3...
Hilo4 - contador: 12
Fin Hilo4...
Terminamos el programa...
```


B. *Wait* y *Notify*

→ Para hacer diferentes operaciones sobre el mismo objeto

Con las regiones críticas permitimos que varios hilos realicen la misma operación de forma sincronizada. Sin embargo, cuando pretendemos que los hilos lleven a cabo distintas operaciones sobre el mismo objeto de forma sincronizada y colaborativa, debemos utilizar otro tipo de métodos.

Con los métodos *wait()*, *notify()* y *notifyAll()* vamos a ser capaces de limitar el acceso a un objeto cuando no se den las condiciones para poder seguir ejecutándose, y de permitir su ejecución cuando estas condiciones se cumplan. La relación entre estos métodos se muestra en la figura 7.10. El objeto que controla los hilos que acceden a él mediante estos mecanismos se conoce con el nombre de **monitor**.

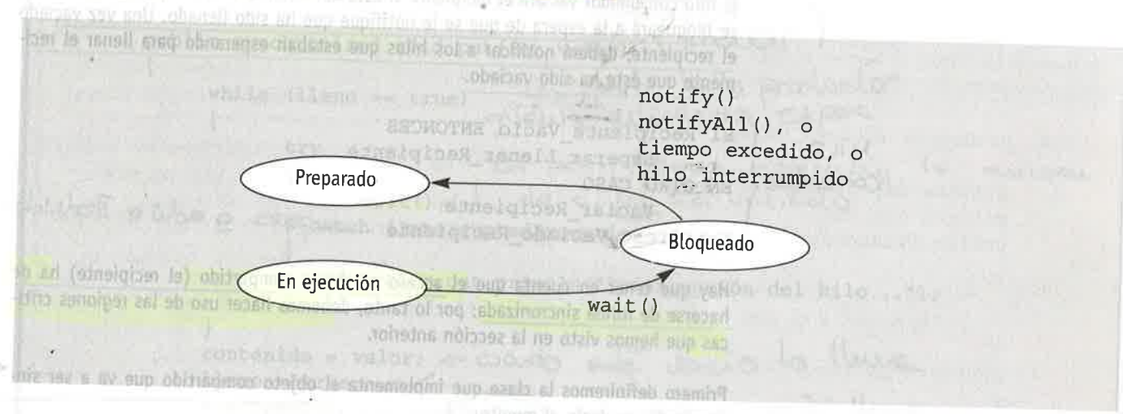


Fig. 7.10. *Cuando un hilo invoca su método* wait(), *queda bloqueado a la espera de que le llegue un evento, que puede ser la invocación al método* notify() *o al método* notifyAll(), *la superación del tiempo especificado en el parámetro del método* wait() *o la interrupción del hilo.*

Para introducir estos métodos vamos a abordar un problema de sincronización típico en informática como es el del productor y el consumidor.

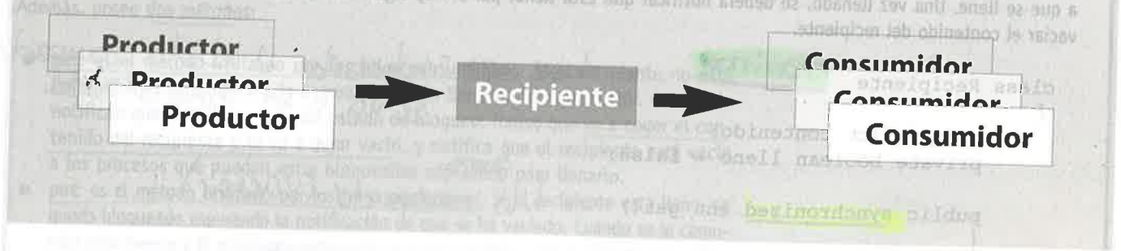


Fig. 7.11. *Esquema de aplicación productores-consumidores.*

Los métodos wait() y notify() han de estar incluidos en un método synchronized.

En nuestro problema existen uno o varios hilos productores, encargados de llenar un recipiente. Un productor llenará el recipiente siempre y cuando esté vacío; si el recipiente está lleno, debe bloquearse. El hilo productor saldrá del estado de bloqueo cuando se le notifique que el recipiente está vacío. Por otra parte, el hilo productor debe notificar a los hilos bloqueados que esperan a que se llene el recipiente que éste se ha llenado.

HILO PRODUCTOR
```
SI Recipiente_Lleno ENTONCES
        Esperar_Vaciar_Recipiente
EN OTRO CASO
        Llenar_Recipiente
Notificar_Llenado_Recipiente
```
→ para q actúe Consumidor

El hilo consumidor vaciará el recipiente si está lleno. En el caso de que no esté lleno, se bloqueará a la espera de que se le notifique que ha sido llenado. Una vez vaciado el recipiente, deberá notificar a los hilos que estaban esperando para llenar el recipiente que éste ha sido vaciado.

HILO CONSUMIDOR
```
SI Recipiente_Vacío ENTONCES
        Esperar_Llenar_Recipiente
EN OTRO CASO
        Vaciar_Recipiente
Notificar_Vaciado_Recipiente
```
→ para q actúe Productor

Hay que tener en cuenta que el acceso al objeto compartido (el recipiente) ha de hacerse de forma sincronizada; por lo tanto, debemos hacer uso de las regiones críticas que hemos visto en la sección anterior.

Primero definiremos la clase que implementa el objeto compartido que va a ser sincronizado, es decir, el monitor.

Monitor

Clase en Java que implementa un monitor. El monitor se compone de un atributo Recipiente donde se almacena un valor entero. Debe contar con dos métodos para almacenar y descargar el contenido de forma sincronizada. Si el recipiente no se ha vaciado, habrá que esperar a que un hilo consumidor lo vacíe. Una vez vaciado, se debe notificar que está vacío, por si hay algún hilo productor bloqueado a la espera de llenarlo. Si el recipiente no se ha llenado, un hilo consumidor deberá esperar a que se llene. Una vez llenado, se deberá notificar que está lleno, por si hay algún hilo consumidor bloqueado para poder vaciar el contenido del recipiente.

```
class Recipiente
{
    private int contenido;
    private boolean lleno = false;

    public synchronized int get()
    {
        while (lleno == false)
```
MONITOR → clase que implementa el objeto compartido y va a ser sincronizado.
para → hilo consumidor (vaciar)
mientras esté vacío espera


```
        {
            try
            {
                wait();
            } catch (InterruptedException e)
            {
                System.out.println("Interrupción del hilo...");
            }
        }
        lleno = false;
        notifyAll();
        return contenido;
    }

    public synchronized void put(int valor)
    {
        while (lleno == true)
        {
            try
            {
                wait();
            } catch (InterruptedException e)
            {
                System.out.println("Interrupción del hilo...");
            }
        }
        contenido = valor;
        lleno = true;
        notifyAll();
    }
}
```
Al no estar lleno se queda en espera de q. le avisen de que está lleno.
cuando está lleno b vacío.
→ notifica q recipiente = vacío
(llenar)
→ para hilo productor mientras esté lleno espera
Al estar lleno espera q le avisen de q. se ha vaciado
cuando está vacío lo llena
→ notifica al resto q está lleno.

El monitor tiene dos atributos:

- **contenido**: objeto donde vamos a almacenar el contenido del recipiente.
- **lleno**: valor lógico que nos indica el estado del recipiente.

Además, posee dos métodos:

- ***get***: es el método utilizadó por los hilos consumidores. Si el recipiente no está lleno, el hilo se bloquea a la espera de que lo llene un hilo productor. Cuando le notifican que está lleno, sale del estado de bloqueo, indica que va a coger el contenido del recipiente y lo va a dejar vacío, y notifica que el recipiente está vacío a los procesos que puedan estar bloqueados esperando para llenarlo.
- ***put***: es el método utilizado por los hilos productores. Si el recipiente está lleno, se queda bloqueado esperando la notificación de que se ha vaciado. Cuando se le comunica este evento y el recipiente está vacío, lo llena y notifica que está lleno a los hilos que puedan estar bloqueados a la espera de que se llenara.

Debemos reflejar aquí la diferencia principal entre los métodos *wait()* y *sleep()*. En los dos métodos del monitor, cuando invocamos el método *wait()*, el hilo se bloquea y el control del monitor queda liberado para que otro hilo pueda llamar a *put()* o *get()* (según donde se bloquee). Sin embargo, si usamos el método *sleep()*, el hilo se bloquea, pero el monitor no se libera, lo que impide que pueda ser usado por otro hilo.

El código de la clase *Hilo Productor* es el siguiente.

Productor

Clase que implementa el productor. El método *run()* de la clase debe invocar el método *put()* de la variable compartida. Después de llenar el recipiente, se bloquea durante un tiempo aleatorio.

```
class Productor extends Thread
{
      private Recipiente rec;
      private int num;
      public Productor(Recipiente p_rec, int p_num)
      {
            rec = p_rec;
            this.num = p_num;
      }
      public void run()
      {
            for (int i = 0; i < 10; i++)
            {
                  rec.put(i);
                  System.out.println("Productor "
                     + this.num + " pone: " + i);
                  try
                  {
                        sleep((int)(Math.random() * 100));
                  } catch (InterruptedException e)
                  {
                    System.out.println("Interrupción del hilo..." );
                  }
            }
      }
}
```


La clase *Hilo Productor* tiene dos atributos:

- **num:** nos sirve como identificador del hilo en el caso de que haya más de un hilo productor. Su valor se asigna en el constructor de la clase.
- **rec:** almacena la referencia del recipiente común. Esta referencia se transfiere a la clase a través del constructor de la clase.

El método *run()* del hilo sólo invoca al método *put()* del recipiente para llenarlo y espera un tiempo determinado antes de volver a llenar el recipiente.

El código de la clase *Hilo Consumidor* es el siguiente.

Consumidor

Clase que implementa el consumidor. El método *run()* de la clase debe invocar el método *get()* de la variable compartida. Después de vaciar el recipiente, se bloquea durante un tiempo aleatorio.

```
class Consumidor extends Thread {
       private Recipiente rec;
       private int num;
       public Consumidor(Recipiente p_rec, int p_num)
       {
             rec = p_rec;
             this.num = p_num;
       }
       public void run()
       {
             int val = 0;
             for (int i = 0; i < 10; i++)
             {
                   val = rec.get();
                   System.out.println("Consumidor "
                      +this.num+" toma: "+val);
                   try
                   {
                         sleep((int)(Math.random() * 100));
                   } catch (InterruptedException e)
                   {
                     System.out.println("Interrupción del hilo..." );
                   }
             }
       }
}
```

El hilo consumidor tiene los mismos atributos que el hilo productor. Sin embargo, en el método *run()*, el hilo consumidor invoca el método *get()* del recipiente para vaciarlo (línea 14) y se bloquea durante unos segundos (línea 19).

Por último, el programa principal deberá crear un objeto de tipo Recipiente y los productores y consumidores que deseemos. En nuestro caso sólo vamos a crear un hilo Productor y un hilo Consumidor:

Programa principal

Programa principal que declara una variable compartida utilizada como Recipiente, un hilo Productor y otro Consumidor. El programa principal inicia los dos hilos.

```
public class ProdConsTest
{
      public static void main(String[] args)
      {
            Recipiente rec = new Recipiente();
            Productor p1 = new Productor(rec, 1);
            Consumidor c1 = new Consumidor(rec, 1);
            p1.start();
            c1.start();
      }
}
```

7.7 Asignación de prioridades a los hilos

Mediante el método *setPriority(int prioridad)* se puede establecer una prioridad a un hilo. Esta prioridad es un entero comprendido en el rango 10 *(MAX_PRIORITY)* – 1 *(MIN_PRIORITY)*. Cuando creamos un hilo y no le asignamos ninguna prioridad, éste posee una prioridad de 5 *(NORM_PRIORITY)*. Por **prioridad** entendemos la prelación a

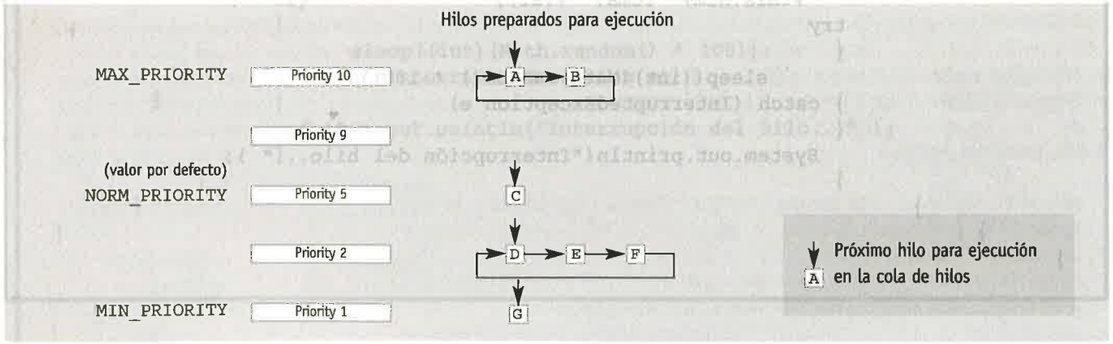


Fig. 7.12. *Colas de hilos preparados-para-ejecución asociadas a cada uno de los niveles de prioridad en Java. En este caso, el hilo* A *es el próximo que pasará a ejecutarse.*

la hora de seleccionar un hilo para pasarlo al estado de en-ejecución por parte del planificador de hilos. Para cada una de las prioridades existe una cola de hilos asociada. El planificador de hilos atiende a los hilos que se encuentran en la cola de máxima prioridad. Si no hay hilos en ella, baja a la siguiente de menor prioridad, y así sucesivamente hasta llegar a la que tiene menos prioridad.

En el siguiente ejemplo comprobaremos el funcionamiento de las prioridades en los hilos. Se trata de un programa en Java con tres hilos, a los que asignamos distintas prioridades. Aunque ponemos preparados-para-ejecución a todos los hilos a la vez, el planificador de hilos dará tiempo de proceso a los que tengan máxima prioridad.

Ejemplo práctico

8 Programa en Java que crea tres hilos, a los que llamaremos *tortuga*, *liebre* y *guepardo*, para comprobar cómo funciona el método *setPriority()*. Estos tres hilos tienen distintas prioridades e imprimen su nombre treinta veces en la salida estándar y se bloquean para dar paso a otros hilos. El hilo de nombre *tortuga* debe tener menos prioridad que el hilo *liebre*, y éste, menos que *guepardo*.

```
class Animal extends Thread
{
    String nombre;

    public Animal(int prioridad, String nombre)
    {
        this.nombre = nombre;
        setPriority(prioridad);
    }

    public void run()
    {
        for (int i=0; i<30; i++)
        {
            System.out.print( nombre );
            yield();
        }
        System.out.println("\nLlega "+nombre+"\n");
    }
}

public class Prioridades
{
    static Animal   tortuga;
    static Animal   liebre;
    static Animal   guepardo;

    public static void main(String argv[])
    {
```

```
        tortuga   = new Animal(4, "T");
        liebre    = new Animal(5, "L");
        guepardo = new Animal(6, "G");
        tortuga.start();
        liebre.start();
        guepardo.start();
    }
}
```

El hilo llamado *guepardo* se encuentra en la cola de hilos de máxima prioridad. Al haber sólo un hilo en esta prioridad, el planificador de hilos irá asignándole tiempo de proceso hasta que termine. Cuando termine, el siguiente hilo en ser atendido será el llamado *liebre*, que tiene más prioridad que el hilo *tortuga*.

```
GGGGGGGGGGGGGGGGGGGGGGGGGGGGGG
Llega G

LLLLLLLLLLLLLLLLLLLLLLLLLLLLLL
Llega L

TTTTTTTTTTTTTTTTTTTTTTTTTTTTTT
Llega T
```

Como se puede comprobar en la ejecución del ejemplo 8, en primer lugar, el hilo llamado *guepardo* escribe treinta veces la letra *G* por la salida estándar. A continuación, el hilo llamado *liebre* escribe treinta veces la letra *L*. Finalmente, el hilo *tortuga* escribe treinta veces la letra *T*.

Ejercicios

A

1. **Realiza un programa en Java que ejecute tres hilos de forma concurrente. Uno de ellos debe sumar los números pares del 1 al 1.000; otro, los impares, y otro, los que terminan en 2 o en 3.**

2. **Realiza un programa en Java que cree un hilo, que a su vez cree otro hilo, y así consecutivamente, hasta un total de cinco hilos. Cada hilo debe presentar su nombre indicando que ha comenzado, crear su hilo hijo y presentar de nuevo su nombre diez veces indicando que se está procesando, esperando un tiempo aleatorio entre 100 y 600 ms entre cada presentación. Cada hilo deberá esperar a que su hijo termine antes de presentar un último mensaje indicando su nombre y que ha terminado.**

3. **Realiza un programa en Java que cree veinte hilos, cada uno de ellos con un número asignado (del 1 al 20) y una prioridad aleatoria y esperar a que terminen. La prioridad deberá ser 6 o 7. La probabilidad de asignar prioridad alta (7) debe ser del 10% y de asignar prioridad baja (6) del 90%. Cada hilo presentará diez veces en pantalla una línea compuesta por tantos guiones como sea su número de hilo.**

4. **Realiza un programa en Java que cree dos hilos de forma que uno de ellos espere a que el otro termine utilizando un semáforo de valor 1, también llamado semáforo binario.**